

Java — brief técnico

Critérios de avaliação, nomes, escopos, tempo de vida e tipos

Integrantes:

Guilherme Coutinho Alexópulos

Pedro Txai Moreira Leão

Yan Capiberibe Yonekura

Yves Yohan Mendonça Boiba

Tese. Java troca parte da concisão por contratos explícitos: a tipagem estática e o escopo léxico dão sinais antecipados de erro, enquanto a JVM automatiza o heap. O preço aparece em verbosidade, boxing/alocações e na necessidade de conhecer regras de inferência e captura.

1 Quatro critérios

Critério	Avaliação de Java	Evidência verificável
Ilegibilidade	A sintaxe explícita ajuda a localizar tipos e efeitos, mas verbosidade e shadowing podem esconder a intenção.	No exemplo <code>EscopoEClosure, x local</code> e <code>EscopoEClosure.x</code> aparecem separados; o compilador aceita ambos, mas a leitura exige atenção.
Redigibilidade	Funções genéricas e interfaces permitem trocar a implementação mantendo a assinatura.	<code>SistemaDeTipos.primeiro</code> funciona para <code>List<T></code> sem duplicar o algoritmo; a assinatura é o contrato que a alteração deve preservar.
Confiabilidade	A checagem estática desloca muitos erros para a compilação; ainda há falhas de lógica e de execução.	JLS §4 define Java como estática e fortemente tipada. Os três arquivos compilam com <code>javac -Xlint:all</code> ; uma atribuição incompatível foi testada como erro de compilação separado.
Custo	A coleta automática reduz <code>free</code> explícito, mas não torna alocações, boxing ou pausas gratuitas.	<code>CustoDeExecucao</code> cria 200.000 strings e mediu 29 ms nesta execução; o número varia, portanto serve como medição do ambiente, não como constante da linguagem.

Decisões de projeto. (1) nomear campos e locais sem colisões; (2) usar interfaces e funções pequenas como contratos; (3) medir alocações e tempo em vez de prometer desempenho. Essas decisões reduzem ambiguidade e tornam o custo discutível com dados.

2 Modelo de nomes, escopos e tempo de vida

Como Java organiza o programa. Java separa nomes locais, parâmetros, campos, classes e pacotes. Essa organização é lexical e explícita: o lugar onde uma declaração aparece determina onde ela pode ser usada e reduz ambiguidades antes da execução.

Nomes em Java

- Variáveis locais e parâmetros ficam limitados à função ou bloco em que foram declarados.
- Campos pertencem à classe ou ao objeto e podem ser escritos como `Classe.campo` ou `this.campo`.
- Classes, interfaces e pacotes formam nomes maiores, organizados por arquivos e módulos.
- Se um nome local esconde um campo, Java permite deixar a diferença visível pela qualificação. No exemplo, `x` e `EscopoEClosure.x` têm valores diferentes.

O diferencial do escopo

- O compilador conhece o escopo lexical antes de rodar o programa; erros de nome e acesso aparecem cedo.
- `private`, `protected` e `public` controlam quem pode acessar um membro, enquanto blocos e funções controlam onde o nome existe.
- Esse modelo favorece manutenção: a alteração de um nome local não precisa afetar o projeto inteiro.
- O custo é a possibilidade de verbosidade: em classes grandes, muitos campos e qualificadores podem dificultar a leitura.

2.1 Closures em Java

Java ganhou closures por meio das lambdas. Uma lambda pode levar consigo uma variável local, mas Java só permite essa captura quando a variável é final ou não muda depois de criada. Esse é um diferencial importante: a linguagem prefere uma regra simples e previsível a permitir qualquer alteração escondida dentro da closure. No arquivo `EscopoEClosure`, base é capturada e o campo `x` é lido; o resultado comprovado é `closure=12`.

Tempo de vida em Java

- Ao sair do bloco, bloco deixa de ser acessível pelo nome.
- Um objeto capturado por uma lambda pode continuar vivo depois que a função que o criou terminou.
- Por isso, sair do escopo não significa que a memória foi liberada naquele instante.

Memória e JVM

- A JVM mantém um heap compartilhado onde ficam objetos e arrays.
- O coletor de lixo recupera objetos que não podem mais ser alcançados; o programador não chama `free`.
- Isso simplifica a programação e evita uma classe comum de erros de ponteiro.
- O custo continua existindo: alocações podem consumir tempo e memória, e recursos externos exigem fechamento explícito.

Diferencial e limite. Java oferece gerenciamento automático de memória, mas não fecha arquivo, socket ou transação por “sair do escopo”. Para esses recursos, o código precisa de um protocolo explícito, como `try-with-resources`.

3 Sistema de tipos

O sistema de tipos de Java

- Java verifica tipos durante a compilação. Uma variável declarada como `String` não recebe um `int` por engano.
- A checagem antecipada ajuda IDEs, refatorações e equipes a encontrar incompatibilidades antes da execução.
- `var` apenas deixa o compilador descobrir o tipo local; depois de compilado, o tipo continua definido.

O diferencial das relações entre tipos

- Java usa relações declaradas: uma classe se torna subtipo por `extends` ou `implements`, não apenas por possuir operações parecidas.
- Interfaces permitem trocar implementações sem alterar quem usa o contrato.
- Essa escolha torna APIs previsíveis, embora exija declarar antecipadamente os contratos que serão compartilhados.

Posição de Java: tipos verificados na compilação, relações declaradas por classes/interfaces e inferência usada para reduzir repetição. Java ganha previsibilidade, mas cobra mais declarações do que linguagens que aceitam qualquer objeto com a mesma operação.

3.1 Inferência e conversões permitidas

Em Java, funções genéricas podem ser escritas uma vez e usadas com vários tipos. O compilador deduz o tipo em chamadas como `primeiro(valores)`, mas continua verificando se a operação é segura. O arquivo executado declara `List<Number> valores = List.of(1, 2.0)` e imprime `primeiro=1`.

Recurso de Java	Como aparece no código	O que isso troca
Conversão numérica	<code>int → long</code> funciona sem cast.	Facilidade para ampliar valores, com possível perda de espaço/performance em escolhas maiores.
Boxing/unboxing	<code>int ↔ Integer</code> em contextos definidos.	Integra primitivos e objetos, mas pode criar objetos e falhar ao desempacotar <code>null</code> .
Interfaces e herança	<code>Integer</code> pode ser usado como <code>Number</code> .	Permite substituição por contrato declarado, sem depender de “parecer ter a mesma operação”.
Casts	Uma conversão mais estreita exige cast explícito.	O risco fica visível no código, mas pode ocorrer <code>ClassCastException</code> em execução.
Genéricos	<code>List<T></code> evita repetir algoritmos.	A reutilização aumenta, mas <code>List<Integer></code> não vira automaticamente <code>List<Number></code> ; o contrato precisa ser compatível.

3.2 Consequência para o projeto

A combinação é um contrato negociável: explicitar tipos aumenta o sinal disponível para ferramentas e revisores, mas declarações e conversões podem alongar o texto. A escolha prática é deixar o compilador inferir o que é local e estável, mantendo explícitos os tipos que formam a fronteira de uma API ou módulo.

4 Exemplos executáveis e auditoria

Os três arquivos estão em exemplos/. A execução comprovada foi feita em OpenJDK 21.12 com:

`javac -encoding UTF-8 -Xlint:all -d build exemplos/*.java` e, depois, `java -cp build NomeDaClasse` para cada arquivo.

4.1 Escopo e closure

```
import java.util.function.IntSupplier;

public class EscopoEClosure {
    static int x = 10;
    static IntSupplier criaLeitura()
    { int base = 2; return () -> base + x; }
    public static void main(String[]
args) {
        int x = 3;
        { int bloco = 4;
System.out.println("local=" + x + ",
bloco=" + bloco); }
        System.out.println("campo=" +
EscopoEClosure.x);
        System.out.println("closure=" +
criaLeitura().getAsInt());
    }
}
```

4.3 Custo medido

```
public class CustoDeExecucao {
    public static void main(String[]
args) {
        final int n = 200_000; long
inicio = System.nanoTime(); long total =
0;
        for (int i = 0; i < n; i++)
        { String item = "item-" + i; total +=
item.length(); }
        long tempoMs =
(System.nanoTime() - inicio) /
1_000_000;
        System.out.printf("n=%d%ntotal-
positivo=%b%ntempo-ms=%d%n", n, total >
0, tempoMs);
    }
}
```

4.2 Tipos e inferência

```
import java.util.List;

public class SistemaDeTipos {
    static <T> T primeiro(List<T> xs)
    { return xs.get(0); }
    public static void main(String[]
args) {
        int inteiro = 7; long ampliado =
inteiro;
        List<Number> valores =
List.of(1, 2.0);
        Number primeiroValor =
primeiro(valores);
        Integer encaixotado = 42; Number
comoReferencia = encaixotado;
        int desempacotado = encaixotado;

System.out.printf("ampliado=%d%ntprimeiro=%s%ntreferencia=%s%ntdesempacotado=%d%n",
ampliado, primeiroValor,
comoReferencia, desempacotado);
    }
}
```

Saída observada. local=3, bloco=4; campo=10; closure=12; ampliado=7; primeiro=1; referencia=42; desempacotado=42; n=200000; total-positivo=true; tempo-ms=29. O último valor varia por máquina e carga.